Topic	Description
Module	Robotics
Module Code	MECH-M-2-ROB-ROB-ILV
Semester	SS 2026
Lecturer	Daniel T. McGuiness, Ph.D
ECTS	4
SWS	1
Lecture Type	ILV
Coursework Name	Individual Assignment
Work	Individual
Suggested Private Study	10 hours
Submission Format	Submission via SAKAI
Submission Deadline	12.07.2026 23:59
Late Submission	Not accepted
Resubmitting Opportunity	No re submission opportunity

> No lecture time is exclusively devoted to the aforementioned assignment.

A portion of the mark for every assignment will be, where applicable, based on style. Style, in this context, refers to organisation, flow, sentence and paragraph structure, typographical accuracy, grammar, spelling, clarity of expression and use of correct IEEE style for citations and references. Students will find *The Elements of Style (3rd ed.)* (1979) by Strunk & White, published by Macmillan, useful with an alternative recommendation being *Economist Style Guide (12th ed.)* by Ann Wroe.

Submission Format and Requirements with LaTeX

Before submitting your work for assessment please follow the following requirements.

1. **The work must be done using LaTeX and no other format will be accepted.** You are allowed to use whatever software you wish to write (i.e., Overleaf, TeXStudio, LyX, ...). The work must also conform to the **MCI** Documentation guidelines (which is in SAKAI). A template is provided to you here which was written by the author of this document.

   > If you are to use **mcidoc** template, set the `document-state` as `Report`. To learn LaTeX, the author has tutorials which can be accessed here.

   Please treat this as a great practice to learn LaTeX as you will need it later.

2. The report (and all its content) **must** be submitted as a `.pdf` along with all the necessary files (`.tex`,`.sty`) to compile it. Submit one (1) `.zip` file containing everything with the following name pattern.

   `[STUDENT-ID]_[STUDENT-NAME]_[STUDENT-SURNAME].zip`

   i.e., **Jason Brigham** with a student ID **2710704051** would send their submission as:

   `2710704051_Jason_Brigham.zip`

   and their submission would be in arranged as the following directory:

   ```
   2710704051_Jason_Brigham/
   ├── tex/
   │   ├── 2710704051_Jason_Brigham.tex, *.pdf, mcidoc.cls, custom.sty, ...
   │   └── figures/
   │       └── .jpg, .png, ...
   └── HW_Content/
       └── src/, include/, .cpp, .bash, .py, ...
   ```

   where `...` are any additional requirement (i.e., `.bib`, `.glo`) you may need for TeX and the `HW_Content` is any content which is relevant to the Homework (i.e., if your HW is about python, this is where all the python scripts would be kept, or where your ROS 2 source files would be, or where your C++ codes would be for a project.)

3. The work must be submitted to SAKAI and **no link to personal repo will be accepted and failure to follow the guideline will result in immediate failure**.

> To make sure your submission conform to given requirements, please test out zipping/unzipping and running any code before submission. Any deviations from the requirements will incur heavy penalties to the final grade.

Question	Maximum Point	Received Point
When Life Gives you ROS, You Programme Turtles	100	
Sum	100	

[Q1] When Life Gives you ROS, You Programme Turtles — 100

This assignment will require you to program a ROS2 package to make multiple robots in `turtlesim` conduct certain actions. To this end your ROS package should do the following:

1. Open a TurtleSim window with its title named (instead of TurtleSim):

   `[STUDENT_ID]-[STUDENT-NAME]-[STUDENT-SURNAME]`

2. The first robot, called `turtle1`, should be spawned in a random location within the environment and then rotate so it is looking at point wall which it is furthest away. Then the robot should print the message (`Furthest point detected!`) to the terminal and will move to that point.

> If a robot is spawned in such a way as to be in between to points, then the robot must pick one of them randomly.

1. Once `turtle1` touches the point, a second turtle, called `turtle2`, will be spawned at a random position within the `TurtleSim` environment.
2. `turtle2` will then rotate to find the point on the wall which it is furthest away and will move there. However, now `turtle1` will now move to follow `turtle2` as `turtle2` is trying to reach its destination.
3. Once `turtle2` has reached its destination, `turtle3` will be spawned in a random location and will rotate to find the point on the wall which it is furthest away. Once `turtle3` has started to move, now `turtle2` will follow `turtle3` and `turtle1` will continue to follow `turtle2`.

> This means when `turtle5` is trying to reach its destination, `turtle4` will be following `turtle5`, which itself is followed by `turtle3`, which is followed by `turtle2`, which is followed by `turtle1`.

4. These actions will continue until `turtle6` is spawned which then the package writes to the terminal.

   `There are too many turtles... Program will terminate...`

5. Then the program terminates and prints out information about how much distance each turtle has covered during the runtime of the simulation. An example of this is shown later in this document.

6. This package will be submitted to SAKAI with a **bash script** which automates the installation and the packages execution.

A diagram showcasing the requirements are given in **Fig.** 1. Your package must be called `turtlesimAutomata` and will be **automatically** installed using a bash script which does the following:

7. Check if the computer is running Ubuntu 22.04.5 LTS.
8. Check if ROS2 is installed on the computer and if **NOT**, ask the user if they would like to install RO2 Humble.
9. Creates a workspace in `~/ros2\_ws/src`.
10. Move all the necessary package contents of the package title `turtlesimAutomata` to the appropriate folder(s) and directories,
11. Compiles the package using `colcon`,
12. Executes the package,
13. Removes the downloaded package inside the `Downloads` folder.
14. Once the program terminates, the user needs to be let know of how much each robot has moved in total. An example terminal output is as follows:

Sample text

```
The program has finished, The results are as follows:
Turle            x           y
                 ---         ---
turtle1         112.09      75.66
turtle2         86.65       61.32
turtle3         56.12       46.32
turtle4         26.78       23.66
turtle5         12.15       12.11
```

Rules and Requirements of your Assignment

Your code must be executable in a Linux environment without any modification. To be able to do this assignment, you need to have Ubuntu installed with ROS 2 Humble. If you need more information, please consult lecture documentation. Prior to starting the assignment, you should make sure you understand ROS 2 concept and should be familiar with the CLI. In addition, go over all the code samples from ROS 2 tutorials and make sure you understand them thoroughly.

1. Make sure that your code is tidy and well-commented.
2. Your package must be written in Python and must conform to `numpy` documentation, which are given in more detail in **Submission Requirements with Python**.
    a) If you are going to use a code from someone, **cite** the part of the code.

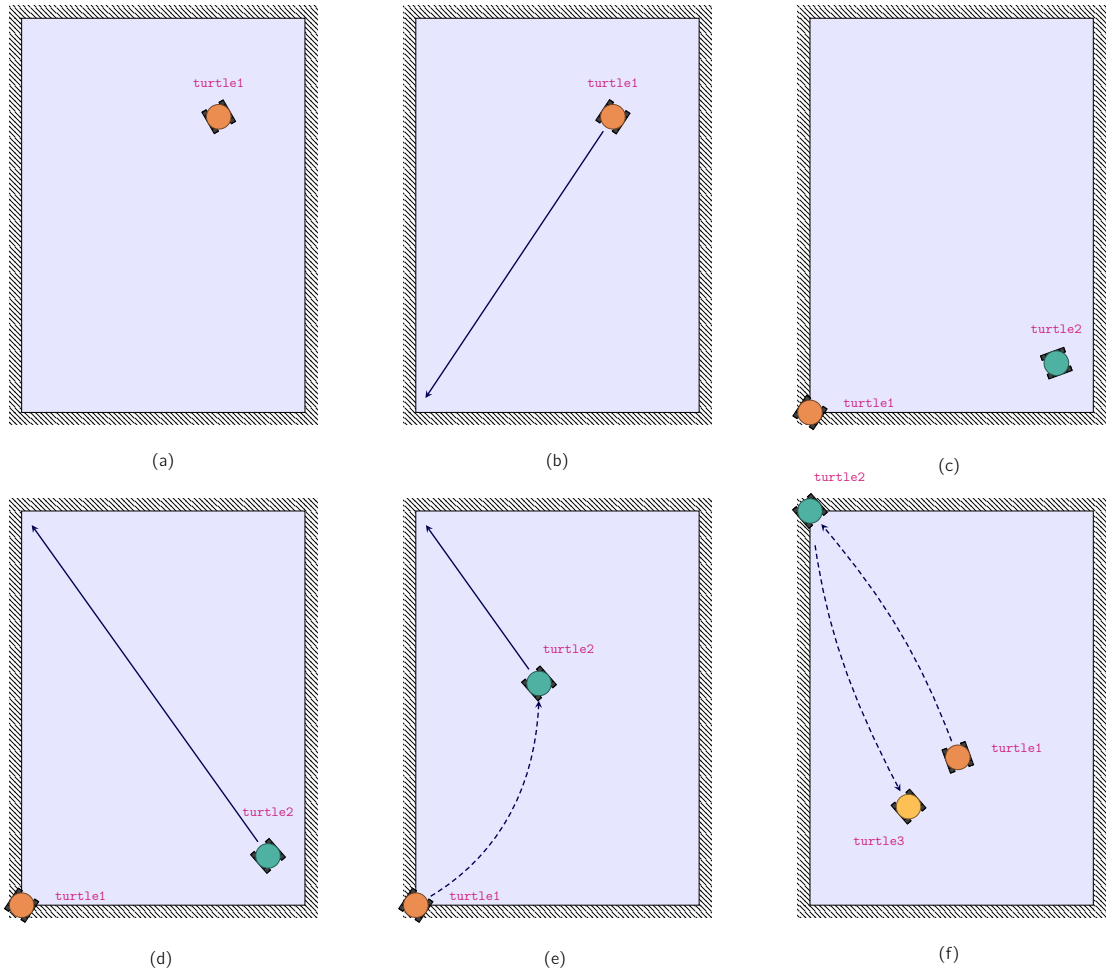

Figure 1: The expected behaviour of the turtles (a) A turtle shall be randomly spawned in the `turtlesim` environment and be named `turtle1` (b) Once spawned, `turtle1` will rotate to find a point on the wall which it is furthest away to and start moving there (c) The moment `turtle1` touches the wall a new turtle, called `turtle2` will be spawned in a random location. `turtle2` will then rotate and find the point on the wall it is furthest away (d) `turtle2` will start to move to the point and `turtle1` will follow `turtle2` using `tf2` listener (f) Once `turtle2` touches the wall `turtle3` will be spawned in a random location and then it will be `turtle2` which will listen to `turtle3` while `turtle1` still listens to `turtle2`. This behaviour of following and spawning will continue until `turtle6` is spawned.

3. This is an individual assignment. All the work you turn in should be yours, and **NOT** done in collaboration with anyone else. If you use any external sources of inspiration, other than official ROS 2 documentation, you must declare it in the ROS2 package manifesto. In addition, the `package.xml` must be filled appropriately.

4. You should hand in everything needed to run your code and must conform to submission guidelines. This means:

    i. Your sources code, adequately commented, so that each distinct part of the code is clearly explained. Your code should only contain code which actually does the required. Please tidy up your code prior to submission as any non-relevant code in the submission shall incur penalty.

Report You are to write a report on your individual assignment which should explain the methods and relevant code snippet used in solving the given problem. A template structure for this assignment could be as follows:

- An introduction to the problem, and the method used to tackle.
- Explanation (i.e, conceptual and mathematical) of the method(s) used in solving the problem. For every significant implementation a code snippet (i.e., `minted`, `listings`) must be provided and should be explained.
- The report should also contain images and relevant terminal outputs in the report.

You should **NOT** hand in executable files or build files, or any other files that can be regenerated. Submission of these files will incur penalties to your submission.

Failure to hand in any source code (i.e., `.py`, `.tex`, `.bash`,...) will result in immediate failure of the assignment.

Your code should run with only one single command, in the correct directory:

```
bash tutlesimAutomata.bash
```

Before submission, please read **Submission Format and Requirements with LaTeX**. Failure to follow will result in the assessment.